Slide 1 · 0:00–0:45

Timoni + CUE, alongside Flux

Start with the tools' responsibilities. CUE describes data and constraints; Timoni uses it to build and package Kubernetes objects. Kustomize transforms existing objects through bases, patches and other operations. Flux continuously reconciles declared state in the cluster. These responsibilities can be combined in a delivery pipeline. An explicit configuration contract describes which inputs are allowed before resources are rendered. A single replica count illustrates constraints and composition, followed by a small kro API and external values supplied through Timoni Runtime.

Timing: 45 seconds. Planned total: 15:00.

Sources: Timoni · Kustomize · Flux

Slide 2 · 0:45–1:30

CUE’s creator: Marcel van Lohuizen

Marcel van Lohuizen created CUE. At Google he was a founding contributor to Borg, where he created the Borg Configuration Language and core tooling. BCL was later generalized into GCL. CUE is a different language informed by that experience. Its design returns to constraint composition and graph unification. The technical connection: configuration can carry allowed values and reusable structure in the same language. The next two slides explain that model with small examples.

Timing: 45 seconds. Planned total: 15:00.

Sources: Marcel’s biography · CUE history

Slide 3 · 1:30–3:00

Types and values share one model

Think of an integer type as the set of all integers, a range as a smaller set, and a concrete value as a singleton. CUE represents them within the same model. Read the expression from inside the parentheses first. The value must be an integer, at least one and at most ten. Ampersand means that every constraint must hold. The vertical bar offers alternatives, and the star marks two as the preferred default.

Walk through the three cases. With no supplied replica value, evaluation selects two. Three satisfies the range and becomes the concrete result. Zero satisfies neither alternative, so evaluation fails. A string such as “three” also fails. The default is a preference, so it does not prevent valid input from selecting three. This expression comes from the included Deployment module. Exporting resources requires concrete values; a range alone does not choose a replica count. Pause on the cases before moving on.

Timing: 90 seconds. Planned total: 15:00.

Sources: Constraints · Defaults

Slide 4 · 3:00–4:30

Composition means satisfying both

The hash introduces a reusable CUE definition. Here Config contains a replica constraint without a default, so supplying a concrete value makes the narrowing operation clear. Combining Config with replicas three gives the structure shown on the right. Reversing those two operands gives the same result. CUE calls this operation unification.

The computer-science connection is intersection: the result must satisfy both sources of information. Unification is commutative, associative and idempotent. Adding the same compatible constraint twice does not change the answer. Supplying zero conflicts with this range; combining replicas three with replicas four conflicts too. That is different from an override operation in which one concrete value replaces another. Defaults support valid choices, as on the previous slide. These algebraic properties describe CUE unification; they do not imply that patches, command-line processing or an entire delivery pipeline can run in arbitrary order.

Timing: 90 seconds. Planned total: 15:00.

Sources: Unification · Definitions · Composition properties

Slide 5 · 4:30–6:00

Transform YAML or constrain inputs

Both examples select three replicas. Kustomize starts from a Deployment with two replicas and transforms that field to three. The replicas transformer is an alternative to a patch for this field. On the right, CUE combines an input with a definition that includes a default and a permitted range. The Timoni module connects values.replicas to the Deployment's replica count.

Now change the input to zero. Kustomize can render zero, which is a valid Kubernetes replica count. It does not encode this example's minimum of one. CUE rejects zero because that minimum is part of the module's contract. This shows where the rule is expressed: schema tools, policies and CI checks can also accompany Kustomize. Both approaches reuse configuration. CUE makes reusable data and constraints part of one language; Kustomize offers familiar object transformations. Use the matching offline examples to inspect both.

Timing: 90 seconds. Planned total: 15:00.

Sources: Kustomize operations · Offline comparison

Slide 6 · 6:00–7:15

Timoni packages CUE modules

Timoni provides a Kubernetes workflow around CUE. A module exposes a configuration interface and connects those inputs to resources. Authors can vendor Kubernetes API or custom-resource schemas so the resource definitions are checked as well as their own configuration rules. This is explicit schema integration: a CUE file alone does not automatically know every Kubernetes API.

The first command renders the local WebApp RGD module. The second runs module validation against its included schema. These local commands do not contact Kubernetes. Timoni can package CUE sources and included schemas as a versioned OCI artifact; consumers build it with their own values. Its CLI also offers apply and instance inventory. Continuous reconciliation remains a separate responsibility. The small Deployment example omits vendored Kubernetes schemas; the RGD example includes the upstream kro schema. Structural validation is useful evidence, but controller behavior still needs runtime testing.

Timing: 75 seconds. Planned total: 15:00.

Sources: Timoni concepts · Modules · CRD schemas

Slide 7 · 7:15–8:45

Use shared CUE structure for a kro API

kro turns a ResourceGraphDefinition into a Kubernetes API and reconciles each instance's resources. This example is a namespaced WebApp backed by one Deployment. CUE combines shared conventions and the vendored kro 0.9.4 schema, then adds graph-specific fields. Both panels are excerpts; the full module includes the remaining Deployment fields.

Read the right panel from schema to resource. Default two and the range are part of the generated API. A WebApp can supply replicas three; the CEL expression reads three and kro is expected to reconcile three replicas. Changing the API default does not rewrite existing instances.

Timoni preserves the expression as data; kro evaluates it later. The RGD is cluster-scoped; WebApps and their Deployments are namespaced. Runtime behavior remains illustrative. When delivering RGDs through Flux, preserve CEL expressions during Flux substitution; the handoff example explains how.

Timing: 90 seconds. Planned total: 15:00.

Sources: kro v0.9.4 · Complete module, including vendored schema · Flux values handoff

Slide 8 · 8:45–9:45

External values and Flux substitution

Infrastructure outputs can enter Kubernetes as ConfigMaps and Secrets, published by Bicep or CI. Flux's substituteFrom reads those objects after Kustomize has built the manifests. This diagram illustrates one delivery pattern; the object names and values are synthetic.

Bicep can type its outputs, while ConfigMap data is string-valued. The consumer still needs to establish the intended type and any additional rules. A replica count might need to be an integer from one to ten.

CUE can validate producer outputs before publication, or check consumer inputs once they are resolved. Checking an earlier placeholder cannot validate a value inserted later by Flux. Timoni Runtime provides another input path: an explicit CLI step reads cluster objects and binds values into a Bundle. The next slides follow the lookup, binding and contract checks in that path. Flux's substitution and reconciliation remain a separate execution model.

Timing: 60 seconds. Planned total: 15:00.

Sources: Bicep outputs · Flux substitution · Values handoff

Slide 9 · 9:45–11:15

Runtime selects resources and fields

A Runtime definition declares which resources to read and how their fields map to named inputs. Read the first query from left to right: Kubernetes API v1, ConfigMap, namespace demo, object infra-values. Its for mapping evaluates obj.data.replicas and exposes the result as DEMO_REPLICAS. The second query selects the token field from a Secret and names the input DEMO_TOKEN. These names connect the Runtime to the Bundle.

The mapping expressions select concrete field values. A ConfigMap can hold the text three; selecting that field does not yet establish the consumer's integer range. The Bundle's attributes specify parsing, and CUE constraints establish the consumer contract on the following slide. A similar mapping can select other scalar fields from Kubernetes resources.

The complete runtime.cue also supplies API metadata and explicitly selects the minikube context. Those fields are omitted from this query excerpt. Runtime reads happen when the CLI is invoked. A later change to a ConfigMap requires another execution to refresh the inputs. The offline example uses environment variables with the same names, allowing contract checks without Kubernetes API access.

Timing: 90 seconds. Planned total: 15:00.

Sources: Runtime queries and mappings · Complete runtime.cue

Slide 10 · 11:15–13:00

Bundle inputs: bind, then constrain

A Bundle declares module instances and their input values. This excerpt shows three distinct parts. First, `#InfraValues` is the reusable consumer contract: replicas must be an integer from one to ten, and token must be a nonempty string. The complete file also declares the Bundle's API version, name, module URL, module version and target namespace.

Second, the Timoni attributes bind the named Runtime inputs. The number attribute parses `DEMO_REPLICAS` as a CUE number; the string attribute retains `DEMO_TOKEN` as text. The underscore fields hold those inputs separately from the values supplied to the module. They are hidden CUE fields and are not exported as application configuration.

Third, `values` unifies `#InfraValues` with the concrete bindings. Text three from the ConfigMap becomes number three, then satisfies the integer range. Zero parses as a number but conflicts with the minimum. A fraction parses as a number but fails the integer requirement. Nonnumeric text fails at parsing, and an empty token conflicts with the nonempty-string constraint.

The distinction between parsing and the consumer contract matters. Keep the constraints on the consumed values, separate from Runtime injection. The same named contract can be reused with other input sources. Building the referenced module then checks its own interface and resource schemas; this local Bundle check does not fetch that placeholder module.

Timing: 105 seconds. Planned total: 15:00.

Sources: Bundles · Runtime bindings · Hidden fields · Complete bundle.cue

Slide 11 · 13:00–14:15

Check the contract offline

Offline mode supplies Runtime inputs from environment variables instead of querying Kubernetes. The variable names match the Bundle attributes. With replicas three and a nonempty dummy token, bundle vet succeeds. The same contract rejects zero, eleven, fractions and nonnumeric text. Missing replicas, a missing token and an empty token also fail.

These failures happen at different layers: nonnumeric text cannot be parsed as a number, a fraction conflicts with int, and zero conflicts with the minimum. This separation makes the input boundary explicit.

Bundle vet checks this Bundle and its concrete contract. The module URL in the example is a placeholder; module builds and resource-schema checks are separate. The command is an offline validation step and applies nothing.

Runtime is executed by the CLI, so changed source values need a new run. Flux's documented templating workflow delivers rendered manifests and does not automatically invoke Runtime injection. A pipeline can resolve, validate and render inputs before handing manifests to Flux, provided it defines when that pipeline runs. The prepared checks demonstrate input validation; they do not implement that delivery pipeline.

Timing: 75 seconds. Planned total: 15:00.

Sources: Offline Runtime inputs · Timoni and Flux · Reproduce the checks

Slide 12 · 14:15–15:00

Capabilities and operating requirements

CUE combines reusable structure with constraints. Timoni adds modules, schema integration, OCI packaging and explicit CLI execution. Runtime binds external values before module evaluation. These capabilities require maintained interfaces, schemas and an execution trigger when inputs change.

Kustomize provides object transformations; Flux continuously reconciles declared state. Their validation and execution responsibilities can be combined with CUE and Timoni in a delivery workflow. The linked examples cover rendering, composition, a kro API and Runtime contracts. Finish at fifteen minutes; questions follow the talk.

Timing: 45 seconds. Planned total: 15:00.